

Angular & TypeScript

Deep Dive with Real-World Examples

A comprehensive reference covering TypeScript fundamentals, Angular architecture, components, data binding, and more — with coding examples and real-life analogies.

Phase 2	TypeScript — Variables, Functions, Classes, Interfaces, Generics
Phase 3	Angular Introduction — SPA, Architecture, Concepts
Phase 4	Angular Setup — Node.js, npm, CLI, Project Structure
Phase 5	Components — Lifecycle, Structure, Practical Projects
Phase 6	Data Binding — Interpolation, Property, Event & Two-Way

Based on Angular 18+ | Part 1 of 3

Table of Contents

1.	Why TypeScript for Angular?	3
2.	Variables & Data Types	3
3.	Functions in TypeScript	4
4.	Interfaces	5
5.	Classes & Access Modifiers	6
6.	Inheritance	7
7.	Generics & Enums	8
8.	Decorators	9
9.	What is Angular?	10
10.	Angular Architecture	11
11.	Angular Setup & Project Structure	12
12.	Components & Lifecycle Hooks	14
13.	Data Binding — All 4 Types	16

Phase 2 — TypeScript

The language that powers Angular

TypeScript is a **strongly typed superset of JavaScript** developed by Microsoft. Every valid JavaScript file is also a valid TypeScript file, but TypeScript adds optional static typing, interfaces, enums, generics, and decorators — features that make large applications far easier to build and maintain. Angular is written entirely in TypeScript and requires it for all application code.

1. Why TypeScript for Angular?

JavaScript	TypeScript
No type checking	Static type checking
Errors found at runtime	Errors caught at compile time
Hard to maintain at scale	Scalable & maintainable code
No interfaces	Interfaces & Generics
No decorators	Full decorator support
Basic IDE support	Excellent IDE autocomplete & refactoring

■ REAL-WORLD EXAMPLE — IDE Assistance

When you type "user." in VS Code, TypeScript immediately shows all available properties — id, name, email — just like IntelliSense in C# or Java. JavaScript gives you nothing until runtime. In a codebase with 200 files, this saves hours every week.

2. Variables & Data Types

TypeScript provides all of JavaScript's primitive types plus several extras. The key rule: **declare the type next to the variable name**, separated by a colon.

```
// Primitive types
let name: string = "Alice";
const age: number = 25;
let isActive: boolean = true;

// Arrays – two equivalent syntaxes
let scores: number[] = [95, 87, 92];
let names: Array<string> = ["Alice", "Bob"];

// Tuple – fixed length, fixed types
let student: [string, number] = ["Alice", 25];

// Union type – can be one of several types
let id: number | string = 101; // can also be "EMP-101"

// any – opt out of type checking (avoid when possible)
let payload: any = fetchFromServer();

// null / undefined
let middle: string | null = null; // might not exist
```

■ NOTE

Use "const" for values that never change (object references, config), "let" for values that may change. Avoid "var" entirely in TypeScript/Angular projects.

■ REAL-WORLD EXAMPLE — E-commerce App

In an online store you might have: `productId: number | string` (some stores use numeric IDs, others use slugs like "nike-air-max"). Union types model this perfectly without resorting to "any".

3. Functions in TypeScript

Functions in TypeScript must declare the **type of every parameter** and the **return type**. This makes the function's contract explicit and prevents entire categories of bugs.

```
// 1. Basic typed function
function add(a: number, b: number): number {
  return a + b;
}

// 2. Optional & default parameters
function greet(name: string, title: string = "Mr"): string {
  return `Hello, ${title} ${name}!`;
}
greet("Smith");           // "Hello, Mr Smith!"
greet("Smith", "Dr");     // "Hello, Dr Smith!"

// 3. Arrow function – concise, lexical "this"
const multiply = (x: number, y: number): number => x * y;

// 4. Void – function that returns nothing
function logMessage(msg: string): void {
  console.log(`[LOG] ${msg}`);
}

// 5. Rest parameters
function sumAll(...nums: number[]): number {
  return nums.reduce((acc, n) => acc + n, 0);
}
sumAll(1, 2, 3, 4);      // 10
```

■ REAL-WORLD EXAMPLE — Banking App

`calculateInterest(principal: number, rate: number, years: number): number` — TypeScript guarantees no one accidentally passes a string rate like "5%" which would silently produce NaN in pure JavaScript.

■ NOTE

Always declare return types explicitly. When a function's purpose changes, TypeScript will immediately flag callers that no longer receive the expected type.

4. Interfaces

An **interface** is a pure TypeScript construct that defines the *shape* of an object — its property names, types, and whether they are required or optional. Interfaces exist only at compile time; they produce no JavaScript output.

```
// Define the contract
interface Student {
  id: number;
  name: string;
  email?: string; // ? = optional
  readonly rollNo: number; // cannot be changed after creation
}

// Create objects that satisfy the interface
const alice: Student = { id: 1, name: "Alice", rollNo: 101 };
// alice.rollNo = 200; // ERROR: cannot assign to readonly property

// Function that accepts any Student-shaped object
function displayStudent(s: Student): void {
  console.log(`${s.name} – Roll: ${s.rollNo}`);
}

// Extending an interface
interface GraduateStudent extends Student {
  thesis: string;
  advisor: string;
}

// Interface for a function signature
interface MathOperation {
  (a: number, b: number): number;
}
const add: MathOperation = (a, b) => a + b;
```

■ REAL-WORLD EXAMPLE — REST API Response Typing

When your Angular service calls GET /api/users, the server returns JSON. Define interface `User { id: number; name: string; email: string; role: string; }` and type the HTTP response as `Observable<User[]>`. Angular + TypeScript will now ensure you never access a non-existent field like `user.username` by mistake.

■ NOTE

Interfaces vs Classes: use interfaces for data shapes (API responses, form models). Use classes when you need methods, constructors, or Angular dependency injection.

5. Classes & Access Modifiers

TypeScript classes extend JavaScript ES6 classes with **access modifiers** that control visibility of properties and methods from outside the class.

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Modifier' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='Modifier', text Color=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Accessible From' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='Accessible From', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Use Case' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='Use Case', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
public	Anywhere	Default. Public API of the class.
private	Inside the class only	Internal implementation details.
protected	Class + subclasses	Shared logic for inheritance hierarchies.
readonly	Read from anywhere; write only in constructor	Immutable IDs, config values.

```

class BankAccount {
  public accountHolder: string;
  private balance:      number;
  readonly accountNo:   string;

  constructor(holder: string, initial: number, accNo: string) {
    this.accountHolder = holder;
    this.balance       = initial;
    this.accountNo     = accNo;
  }

  deposit(amount: number): void {
    if (amount <= 0) throw new Error("Amount must be positive");
    this.balance += amount;
  }

  getBalance(): number {
    return this.balance; // controlled read access
  }
}

const acc = new BankAccount("Alice", 5000, "HDFC-9021");
acc.deposit(1000);
console.log(acc.getBalance()); // 6000
// acc.balance = 999999;      // ERROR: private
// acc.accountNo = "FAKE";    // ERROR: readonly

```

■ REAL-WORLD EXAMPLE — Banking Real-World Analogy

balance is private, just like a real bank account — you can deposit/withdraw through the teller (methods), but you cannot directly edit the balance in the database. accountNo is readonly because it never changes after creation.

6. Inheritance

Inheritance lets a class (*child*) reuse and extend the behaviour of another class (*parent*). The child gets all public and protected members of the parent and can **override** methods to provide specialised behaviour.

```
// Parent (base) class
class Person {
  constructor(public name: string, public age: number) {}

  introduce(): string {
    return `Hi, I'm ${this.name}, aged ${this.age}`;
  }
}

// Child class – inherits from Person
class Teacher extends Person {
  constructor(name: string, age: number, public subject: string) {
    super(name, age); // MUST call parent constructor first
  }

  introduce(): string { // override
    return `${super.introduce()}, I teach ${this.subject}`;
  }
}

class Student extends Person {
  constructor(name: string, age: number, public gpa: number) {
    super(name, age);
  }

  introduce(): string {
    return `${super.introduce()}, GPA: ${this.gpa}`;
  }
}

const t = new Teacher("Ms Sharma", 34, "Mathematics");
const s = new Student("Raj", 20, 9.1);
console.log(t.introduce());
// "Hi, I'm Ms Sharma, aged 34, I teach Mathematics"
console.log(s.introduce());
// "Hi, I'm Raj, aged 20, GPA: 9.1"
```

■ REAL-WORLD EXAMPLE — School Management System

Person is the base. Teacher, Student, and Principal all extend it. Each shares name and age via the parent, but adds its own properties (subject, gpa, department). Angular's own classes follow this pattern — HttpClient, Router, etc. are all regular TypeScript classes.

■ NOTE

Always call `super()` as the first statement in a child constructor. Forgetting it causes a runtime error.

7. Generics & Enums

Generics allow you to write functions and classes that work with *any* type while still being type-safe. Think of them as "type parameters" for your code.

```
// Generic function – T is the type parameter
function getFirst<T>(arr: T[]): T {
  return arr[0];
}

getFirst<number>([10, 20, 30]); // 10 (TypeScript knows it's number)
getFirst<string>(["Alice", "Bob"]); // "Alice"

// Generic interface – used in Angular services
interface ApiResponse<T> {
  data: T;
  status: number;
  message: string;
}

type UserResponse = ApiResponse<User>;
type ProductResponse = ApiResponse<Product[]>;

// Enum – named constants, no magic strings
enum UserRole {
  Admin = "ADMIN",
  Student = "STUDENT",
  Teacher = "TEACHER",
}

enum OrderStatus {
  Pending = "PENDING",
  Shipped = "SHIPPED",
  Delivered = "DELIVERED",
  Cancelled = "CANCELLED",
}

let role: UserRole = UserRole.Admin;
if (role === UserRole.Admin) {
  showAdminDashboard();
}
```

■ REAL-WORLD EXAMPLE — E-commerce Order Tracking

OrderStatus enum replaces raw strings scattered across the codebase. Instead of comparing `order.status === "shipped"` (typo-prone), you write `order.status === OrderStatus.Shipped`. If the value ever changes, update one enum entry and TypeScript highlights every affected usage instantly.

8. Decorators — Angular's Magic

A **decorator** is a special TypeScript function that attaches metadata or modifies a class, method, property, or parameter. In Angular, decorators are everywhere and are the primary way the framework understands your code.

```
// @Component — tells Angular: "this class is a UI component"
@Component({
  selector:    "app-product-card",    // HTML tag to use
  templateUrl: "../product-card.component.html",
  styleUrls:   ["../product-card.component.scss"]
})
export class ProductCardComponent { }

// @Injectable — makes a class available for dependency injection
@Injectable({ providedIn: "root" })
export class ProductService { }

// @Input / @Output — property decorators for component communication
@Input() product!: Product;           // receive data from parent
@Output() addToCart = new EventEmitter<Product>(); // emit to parent

// @HostListener — react to DOM events on the host element
@HostListener("mouseenter")
onHover() { this.highlighted = true; }

// @ViewChild — get reference to a child element/component
@ViewChild("searchInput") searchInput!: ElementRef;
```

■ REAL-WORLD EXAMPLE — Custom Decorator

You can write your own decorators. For example, a `@Log()` decorator that automatically logs method calls: every time `addProduct()` runs, it prints "addProduct called with [args]". Angular's own decorators work the same way internally.

■ NOTE

Decorators run at class definition time (not instantiation time). They are Angular's substitute for XML configuration files found in frameworks like Spring.

Phase 3 — Angular Introduction

What is Angular and how does it work?

9. What is Angular?

Angular is a **platform and framework** for building client-side single-page applications (SPAs) in HTML and TypeScript. It is developed and maintained by Google and powers applications like Gmail, Google Cloud Console, and Forbes.

Single Page Application	The browser loads one HTML page. Angular intercepts navigation and swaps components without a full reload, giving a native-app feel.
Component-Based Architecture	The entire UI is split into small, reusable, self-contained components — like LEGO bricks that snap together.
Two-Way Data Binding	The model (TypeScript class) and the view (HTML template) stay automatically in sync, eliminating manual DOM manipulation.
Dependency Injection	Services are injected into components rather than created inside them. Makes code testable and reusable.
TypeScript First	The entire framework is written in TypeScript, giving you compile-time safety across the codebase.
RxJS & Observables	Async operations (HTTP calls, user events) are handled with RxJS Observables — a powerful reactive programming model.

■ REAL-WORLD EXAMPLE — Gmail

When you click Compose in Gmail, Angular does not reload the page. It instantiates a `ComposeComponent`, renders it in the DOM, and wires its form inputs to the component class with two-way binding. When you click Send, the `GmailService` (injected service) posts the email via HTTP — all without a page refresh. This is Angular in production.

10. Angular Architecture

An Angular application is organised into a hierarchy of **NgModules**, each grouping related components, directives, pipes, and services. At runtime, the root module bootstraps the root component which acts as the shell of the application.

Building Block	Decorator	Responsibility
----------------	-----------	----------------

NgModule	@NgModule	Groups components, services, pipes. Root module bootstraps the app. Feature modules enable lazy loading.
Component	@Component	The basic building block. Has a TypeScript class (logic), HTML template (view), and CSS (style).
Template	HTML + Angular syntax	Extends HTML with directives (*ngIf, *ngFor), binding syntax, and pipes.
Directive	@Directive	Extends HTML. Structural (*ngIf, *ngFor) change the DOM. Attribute ([ngClass]) change appearance.
Service	@Injectable	Holds shared business logic. Injected into components via the DI system.
Pipe	@Pipe	Transforms template output. {{ price currency:"INR" }} formats numbers as currency.
Router	RouterModule	Maps URL paths to components. Enables navigation without page reloads.
HttpClient	HttpClientModule	Makes HTTP requests. Returns Observables for async data.

Phase 4 — Angular Setup

Installing Node.js, Angular CLI, and creating your first project

11. Installation & Project Structure

Angular requires Node.js (v18+) and npm. The Angular CLI (**ng**) is the command-line tool that creates projects, generates code, runs the dev server, and builds production bundles.

```
# Step 1 — Install Node.js from https://nodejs.org
node --version    # v18.x or higher
npm --version     # v9.x or higher

# Step 2 — Install Angular CLI globally
npm install -g @angular/cli

# Step 3 — Create a new project
ng new my-angular-app
#   Would you like to add Angular routing? YES
#   Which stylesheet format? SCSS

# Step 4 — Run the dev server
cd my-angular-app
ng serve
#   Open http://localhost:4200

# Useful CLI commands
ng generate component home      # create a component
ng generate service product    # create a service
ng build --configuration=production # production build
ng test                        # unit tests
```

Project Structure Explained

Path	Purpose
<code>src/app/</code>	Main application code — components, services, modules
<code>app.component.ts</code>	Root component class (TypeScript logic)
<code>app.component.html</code>	Root component template (HTML)
<code>app.module.ts</code>	Root NgModule — declares all components
<code>angular.json</code>	CLI workspace configuration (build, test settings)
<code>package.json</code>	npm dependencies and build scripts
<code>tsconfig.json</code>	TypeScript compiler configuration

<code>src/assets/</code>	Static files: images, fonts, icons
<code>src/index.html</code>	Single HTML entry point (SPA host page)
<code>src/environments/</code>	<code>environment.ts</code> (dev) / <code>environment.prod.ts</code> (prod)

■ **NOTE**

Never manually edit files inside `node_modules/`. Run "ng generate" for new files instead of creating them manually — the CLI automatically registers components in `app.module.ts`.

Phase 5 — Components

The fundamental building block of every Angular UI

12. Components

A **component** controls a piece of the user interface. Every Angular app is a tree of components rooted at AppComponent. A component is defined by the `@Component` decorator applied to a TypeScript class. The CLI generates all four required files automatically.

```
# Generate with the CLI
ng generate component products/product-card
# Creates:
# src/app/products/product-card/
#   product-card.component.ts    - class + @Component decorator
#   product-card.component.html - template
#   product-card.component.scss - styles (scoped to this component)
#   product-card.component.spec.ts - unit tests

// product-card.component.ts
import { Component, Input, Output, EventEmitter } from "@angular/core";
import { Product } from "../../models/product.interface";

@Component({
  selector: "app-product-card",
  templateUrl: "./product-card.component.html",
  styleUrls: ["./product-card.component.scss"]
})
export class ProductCardComponent {
  @Input() product!: Product;
  @Output() addToCart = new EventEmitter<Product>();

  onAddToCart(): void {
    this.addToCart.emit(this.product);
  }
}
```

```

<!-- product-card.component.html -->
<div class="card">
  <img [src]="product.imageUrl" [alt]="product.name">
  <h3>{{ product.name }}</h3>
  <p class="price">{{ product.price | currency:"INR" }}</p>
  <button (click)="onAddToCart()">Add to Cart</button>
</div>

<!-- Usage in parent template -->
<app-product-card
  *ngFor="let p of products"
  [product]="p"
  (addedToCart)="handleAddToCart($event)">
</app-product-card>

```

Component Lifecycle Hooks

Angular calls a set of lifecycle hook methods at defined moments in a component's life — creation, change detection, and destruction.

Hook	When it fires	Common Use
<code>ngOnChanges()</code>	FIRST to fire. Called every time an <code>@Input</code> property changes. Receives a <code>SimpleChanges</code> object with old and new values.	Recalculate derived data when input changes.
<code>ngOnInit()</code>	Called ONCE after the first <code>ngOnChanges</code> . Component is fully initialised.	Fetch initial data (HTTP calls), subscribe to route params.
<code>ngDoCheck()</code>	Called on every change-detection cycle. Expensive — use sparingly.	Custom change detection for complex objects.
<code>ngAfterContentInit()</code>	Called once after projection is complete.	Access projected content with <code>@ContentChild</code> .
<code>ngAfterViewInit()</code>	Called once after the component's view (and child views) are fully rendered.	Access DOM elements with <code>@ViewChild</code> .
<code>ngOnDestroy()</code>	Called just before Angular destroys the component.	Unsubscribe from Observables, clear timers.

```
import { Component, OnInit, OnDestroy } from "@angular/core";
import { Subscription } from "rxjs";

@Component({ selector: "app-dashboard", templateUrl: "./dashboard.component.html" })
export class DashboardComponent implements OnInit, OnDestroy {
  private sub!: Subscription;
  stats: DashboardStats | null = null;

  constructor(private statsService: StatsService) {}

  ngOnInit(): void {
    // Fetch data when component loads
    this.sub = this.statsService.getStats().subscribe(data => {
      this.stats = data;
    });
  }

  ngOnDestroy(): void {
    // Prevent memory leak — unsubscribe when component is destroyed
    this.sub.unsubscribe();
  }
}
```

■ REAL-WORLD EXAMPLE — Dashboard Widget

When you navigate to the Dashboard page, `ngOnInit` fires and triggers an HTTP request for live analytics data. When you navigate away, `ngOnDestroy` cancels the subscription — without this, the subscription would keep firing in the background, causing a memory leak and potentially updating a component that no longer exists in the DOM.

Phase 6 — Data Binding

How Angular connects TypeScript class properties to the HTML template

13. All 4 Types of Data Binding

Data binding is the mechanism that keeps the component class (the model) and the HTML template (the view) in sync. Angular provides four distinct binding syntaxes, each serving a different direction of data flow.

<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Type' 'frags': [ParaFrag (__tag__='b', bold=1, fontName= ='Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='Type', textC olor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Syntax' 'frags': [ParaFrag(__tag__='b' , bold=1, fontName= 'Helvetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='Syntax', textColor=Color(1,1,1, 1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Direction' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Hel vetica-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='Direction', textCol or=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>	<pre>Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Use For' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helveti ca-Bold', fontSize=10, greek=0, italic=0, link=[], rise=0, text='Use For', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph</pre>
Interpolation	{{ value }}	Class → Template	Display text/values
Property Binding	[property]="expr"	Class → Template	Bind DOM properties
Event Binding	(event)="handler()"	Template → Class	Handle user actions
Two-Way Binding	[(ngModel)]="prop "	Class ↔ Template	Form inputs, live sync

① Interpolation — Class → Template

The double-curly-brace syntax `{{ expression }}` evaluates a TypeScript expression and inserts the result as a string into the HTML.

```
// Component class
export class ProfileComponent {
  userName: string = "Alice";
  cartCount: number = 3;
  discount: number = 0.15;

  get formattedDiscount(): string {
    return (this.discount * 100) + "% off";
  }
}

<!-- Template -->
<h1>Welcome, {{ userName }}!</h1>
<p>Items in cart: {{ cartCount }}</p>
<p>{{ formattedDiscount }}</p>
<p>{{ 5 * 20 }}</p> <!-- 100 - expressions work too -->
<p>{{ "Hello".toUpperCase() }}</p> <!-- HELLO -->
```

■ REAL-WORLD EXAMPLE — E-commerce Header

The top nav of an Amazon-style app shows "Hello, Alice" and a cart badge "3". Both are driven by interpolation: `{{ currentUser.name }}` and `{{ cart.itemCount }}`. When the user adds an item, Angular re-renders automatically.

② Property Binding — Class → Template

Square brackets `[property]="expression"` bind a DOM property to a TypeScript expression. This is different from attributes — you are setting the JavaScript property of the DOM element.

```
// Component class
export class ProductComponent {
  imageUrl: string = "assets/shoe.jpg";
  isOutOfStock: boolean = false;
  productName: string = "Nike Air Max";
  discount: number = 20;
}

<!-- Template -->
<img [src]="imageUrl" [alt]="productName">
<button [disabled]="isOutOfStock">
  {{ isOutOfStock ? "Out of Stock" : "Add to Cart" }}
</button>
<div [class.sale-badge]="discount > 0">
  {{ discount }}% OFF
</div>

<!-- Binding to style property -->
<div [style.background-color]="product.color">...</div>
```

■ REAL-WORLD EXAMPLE — Product Listing Page

When the backend marks a product as `out_of_stock: true`, Angular sets `[disabled]="true"` on the Add-to-Cart button, making it unclickable. No manual DOM manipulation — just a TypeScript boolean.

③ Event Binding — Template → Class

Parentheses **(event)="handler(\$event)"** listen to DOM events and call a method on the component class. The special **\$event** variable carries the native DOM event object.

```
// Component class
export class SearchComponent {
  query: string = "";
  results: any[] = [];

  onSearch(): void {
    this.searchService.search(this.query).subscribe(r => {
      this.results = r;
    });
  }

  onKeyUp(event: KeyboardEvent): void {
    if (event.key === "Enter") this.onSearch();
  }

  onFileChange(event: Event): void {
    const input = event.target as HTMLInputElement;
    const file = input.files?.[0];
    // process file upload
  }
}

<!-- Template -->
<input (keyup)="onKeyUp($event)" [(ngModel)]="query">
<button (click)="onSearch()">Search</button>
<input type="file" (change)="onFileChange($event)">
```

■ REAL-WORLD EXAMPLE — Search Bar

Google's search box fires an event on every keystroke. In Angular: `(input)="onInput($event)"` calls the handler, which debounces the query and fires an HTTP request. The component class stays in full control — the template is just the "trigger".

④ Two-Way Binding — Class ↔ Template

[(ngModel)] combines property binding and event binding into a single shorthand — often called "banana in a box". When the class property changes, the input reflects it; when the user types, the class property updates immediately.

```
// app.module.ts — REQUIRED: import FormsModule
import { FormsModule } from "@angular/forms";
@NgModule({
  imports: [BrowserModule, FormsModule],
  ...
})

// Component class
export class RegistrationComponent {
  firstName: string = "";
  lastName: string = "";
  email: string = "";
  password: string = "";

  get fullName(): string {
    return `${this.firstName} ${this.lastName}`.trim();
  }

  onSubmit(): void {
    console.log("Registering:", this.email);
  }
}

<!-- Template -->
<h2>Hello, {{ fullName }}!</h2>
<input [(ngModel)]="firstName" placeholder="First Name">
<input [(ngModel)]="lastName" placeholder="Last Name">
<input [(ngModel)]="email" placeholder="Email" type="email">
<input [(ngModel)]="password" placeholder="Password" type="password">
<button (click)="onSubmit()">Register</button>
```

■ REAL-WORLD EXAMPLE — Live Form Preview

A registration form shows a live preview card: "Hello, Alice Smith!" updates as the user types their first and last name. This is two-way binding in action — the class properties `firstName` and `lastName` are bound to the inputs, and `fullName` (a getter) reflects the combined value instantly in the heading.

■ NOTE

`[(ngModel)] = [ ] + ( )` combined. You can always split it: `[ngModel]="name"` `(ngModelChange)="name=$event"`. In reactive forms (`FormGroup`), you skip `ngModel` entirely.

Quick Reference Cheat Sheet

TypeScript

<code>let x: type = value;</code>	Variable with type
<code>interface I { p: T }</code>	Object shape / contract
<code>class C { }</code>	Encapsulation + methods
<code>class B extends A { }</code>	Inheritance
<code>function f<T>(a:T):T</code>	Generic function
<code>enum E { A="a" }</code>	Named constants
<code>@Decorator()</code>	Metadata / modification

Angular CLI

<code>ng new app-name</code>	Create project
<code>ng serve</code>	Dev server :4200
<code>ng generate component name</code>	New component
<code>ng generate service name</code>	New service
<code>ng build --prod</code>	Production build
<code>ng test</code>	Run unit tests

Data Binding

<code>{{ value }}</code>	Interpolation
<code>[property]="expr"</code>	Property binding
<code>(event)="handler()"</code>	Event binding
<code>[(ngModel)]="prop"</code>	Two-way binding
<code>[class.active]="bool"</code>	Class binding
<code>[style.color]="expr"</code>	Style binding

Lifecycle Hooks

<code>ngOnInit()</code>	After first render – fetch data
<code>ngOnChanges()</code>	@Input changed
<code>ngAfterViewInit()</code>	DOM fully rendered
<code>ngOnDestroy()</code>	Cleanup subscriptions

You now have a solid foundation in Angular & TypeScript

Topics covered in this guide:

- ✓ TypeScript variables, types & arrays
- ✓ Functions — typed, optional, arrow, rest
- ✓ Interfaces for API response shapes
- ✓ Classes with public / private / protected
- ✓ Inheritance with extends and super()
- ✓ Generics for reusable, type-safe utilities
- ✓ Enums to eliminate magic strings
- ✓ Decorators — @Component, @Input, @Output
- ✓ What Angular is and how it compares to React/Vue
- ✓ Angular architecture — modules, components, services
 - ✓ Setting up Angular CLI and project structure
- ✓ Component lifecycle hooks — ngOnInit to ngOnDestroy
- ✓ All 4 data binding types with real-world examples

Continue with Part 2 → [Directives](#) · [Pipes](#) · [Reactive Forms](#) · [Services](#) · [Routing](#) · [HttpClient](#)